

Introduction to Python

Python is a high-level, general-purpose programming language. It is designed to make programs easy to read and write.

Python uses a simple syntax, so beginners can learn programming concepts without writing many lines of code.

- Python was created by **Guido van Rossum** and first released in 1991.
- Python programs normally use the **.py** file extension.
- Python is interpreted, which means the Python interpreter executes the program.
- Python supports procedural, object-oriented and functional programming.
- Python is used in web development, automation, artificial intelligence, data science, machine learning, testing and scripting.

First program:

```
print("Hello, Python!")
```

The `print()` function displays information on the screen.

Features of Python

- **Simple:** Python syntax is easy to understand.
- **Readable:** Programs look close to normal English.
- **Interpreted:** Code is executed by the interpreter.
- **Portable:** The same program can usually run on Windows, Linux and macOS.
- **Object-oriented:** Python supports classes and objects.
- **Open source:** Python is freely available.
- **Large library:** Python provides many built-in modules and packages.
- **Dynamically typed:** A variable can hold different types of values at different times.
- **Extensible:** Python can work with code written in languages such as C and C++.

Example:

```
x = 10
print(x)
x = "Python"
print(x)
```

Here, the same variable name is used with an integer and later with a string.

Advantages of Python

- **Easy to learn:** The syntax is simple for beginners.
- **Less code:** Many tasks can be completed with fewer lines.
- **Free:** Python can be downloaded and used without a license fee.
- **Large community:** Many learning resources and packages are available.
- **Cross-platform:** Programs can work on different operating systems.
- **Useful libraries:** Libraries are available for web, AI, data analysis, graphics and more.
- **Fast development:** Simple syntax can reduce development time.
- **Easy maintenance:** Readable code is easier to understand and modify.

Example of short Python code:

```
for i in range(1, 6):  
    print(i)
```

The loop prints numbers from 1 to 5 using only a few lines.

Disadvantages of Python

- **Slower execution:** Python is generally slower than compiled languages such as C and C++ for many CPU-intensive tasks.
- **Memory usage:** Python programs can use more memory for some applications.
- **Mobile development:** Python is not a common first choice for native Android or iOS applications.
- **Runtime type errors:** Because Python is dynamically typed, some type problems are found only when the program runs.
- **Large applications:** Careful design is needed when building very large systems.
- **Database-heavy applications:** For some database-intensive workloads, other technologies may be preferred.

These disadvantages do not mean Python is unsuitable. The correct language depends on the requirements of the application.

Python Syntax, Indentation and Comments

Python uses **indentation** to define a block of code. Spaces or tabs must be used consistently.

```
if 10 > 5:  
    print("10 is greater")
```

Comments are notes written in the program. They are ignored during execution.

```
# This is a single-line comment  
print("Hello")
```

Multi-line documentation/comment style:

```
"""  
This is a multi-line string.  
It can be used as documentation.  
"""
```

Rules:

- Statements usually do not need a semicolon.
- Indentation is important after if, for, while, def, class and similar statements.
- Python is case-sensitive: Name and name are different.
- Meaningful variable names make programs easier to understand.

Variables and Constants

A **variable** is a name that refers to a value. Python creates a variable when a value is assigned to it.

```
name = "Ravi"  
age = 20  
marks = 85.5
```

Multiple assignment:

```
a, b, c = 10, 20, 30
```

Swapping values:

```
a = 10  
b = 20  
a, b = b, a
```

Python does not require a separate declaration such as `int age` before assigning a value.

Naming rules:

- A name can contain letters, digits and underscore.
- A name cannot start with a digit.
- Keywords such as `if`, `for` and `class` cannot be used as normal variable names.
- Python has no special constant keyword; uppercase names are commonly used by convention.

Data Types

A data type tells Python what kind of value is stored in a variable.

- **int**: Whole numbers. Example: 25
- **float**: Decimal numbers. Example: 25.5
- **complex**: Complex values. Example: 2 + 3j
- **bool**: True or False
- **str**: Text such as "Python"
- **list**: Ordered and changeable collection
- **tuple**: Ordered and unchangeable collection
- **set**: Unordered collection of unique values
- **dict**: Collection of key-value pairs
- **None**: Represents no value or an absent value

Checking a data type:

```
x = 25
print(type(x))
name = "Python"
print(type(name))
```

Input and Output

The **input()** function reads data from the user. Input is received as a string unless it is converted.

```
name = input("Enter your name: ")  
print("Hello", name)
```

Taking numeric input:

```
age = int(input("Enter age: "))  
print(age)
```

Output using print():

```
name = "Ravi"  
age = 20  
print("Name:", name)  
print("Age:", age)
```

Formatted output using f-string:

```
name = "Ravi"  
age = 20  
print(f"My name is {name} and I am {age} years old.")
```

Operators

Operators are symbols or keywords used to perform operations.

- **Arithmetic:** +, -, *, /, %, //, **
- **Comparison:** ==, !=, >, <, >=, <=
- **Logical:** and, or, not
- **Assignment:** =, +=, -=, *=, /=, //=, %=
- **Membership:** in, not in
- **Identity:** is, is not
- **Bitwise:** &, |, ^, ~, <<, >>

Example:

```
a = 10
b = 3
print(a + b)
print(a > b)
print(a > 5 and b < 5)
print(3 in [1, 2, 3])
```

Decision-Making Statements

Decision-making statements allow a program to choose different actions based on a condition.

if:

```
marks = 75
if marks >= 40:
    print("Pass")
```

if-else:

```
if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

if-elif-else:

```
if marks >= 75:
    print("Distinction")
elif marks >= 40:
    print("Pass")
else:
    print("Fail")
```

Loops and Loop Control

Loops are used to repeat a block of code.

for loop:

```
for i in range(1, 6):  
    print(i)
```

while loop:

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

break: Stops the loop.

```
for i in range(1, 6):  
    if i == 3:  
        break  
    print(i)
```

continue: Skips the current iteration.

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

Strings

A string is a sequence of characters enclosed in single quotes, double quotes or triple quotes.

```
name = "Python"
print(name)
print(name[0])
print(name[-1])
```

Common string operations:

```
text = "python programming"
print(text.upper())
print(text.lower())
print(text.title())
print(len(text))
print(text.replace("python", "Java"))
```

Slicing:

```
word = "Python"
print(word[0:3])
print(word[2:])
print(word[:4])
```

Lists

A list stores multiple values in one variable. Lists are ordered, indexed and changeable.

```
numbers = [10, 20, 30]
print(numbers[0])
numbers.append(40)
print(numbers)
```

Common list methods:

- `append()` – adds an item at the end.
- `insert()` – adds an item at a specified position.
- `remove()` – removes a specified item.
- `pop()` – removes an item using its index.
- `sort()` – sorts the list.
- `reverse()` – reverses the list.

List slicing:

```
a = [10, 20, 30, 40, 50]
print(a[1:4])
```

Tuples, Sets and Dictionaries

Tuple: Ordered collection that cannot normally be changed after creation.

```
t = (10, 20, 30)
print(t[0])
```

Set: Collection of unique values.

```
s = {10, 20, 20, 30}
print(s)
```

Dictionary: Stores information using key-value pairs.

```
student = {"name": "Ravi", "age": 20}
print(student["name"])
student["age"] = 21
```

Useful dictionary methods: keys(), values(), items(), get(), update()

```
print(student.keys())
print(student.values())
```

Functions

A function is a reusable block of code that performs a specific task. Functions reduce repeated code.

Defining and calling a function:

```
def greet():  
    print("Hello")  
  
greet()
```

Function with parameters and return value:

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result)
```

Default parameter:

```
def greet(name="Student"):  
    print("Hello", name)  
  
greet()
```

Functions can have local variables, parameters and return values. Python also supports recursive functions and lambda functions.

Object-Oriented Programming

Object-Oriented Programming (OOP) organizes programs using **classes** and **objects**.

- **Class:** Blueprint or template for creating objects.
- **Object:** A real instance created from a class.
- **Encapsulation:** Keeping related data and methods together and controlling access.
- **Inheritance:** Creating a new class from an existing class.
- **Polymorphism:** One interface or method name can be used with different implementations.
- **Abstraction:** Hiding implementation details and showing the required features.

Class and object syntax:

```
class Student:
    def display(self):
        print("Student details")

s1 = Student()
s1.display()
```

Inheritance and Polymorphism

Inheritance allows a child class to use members of a parent class.

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

d = Dog()
d.sound()
d.bark()
```

Method overriding / polymorphism:

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

for animal in (Dog(), Cat()):
    animal.sound()
```

The same method name **sound()** produces different behavior for Dog and Cat objects.

Exception Handling, File Handling and Modules

Exception handling: Used to handle runtime problems safely.

```
try:
    a = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Program completed")
```

File handling: Files can be opened in modes such as r (read), w (write), a (append).

```
file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

Reading a file:

```
file = open("data.txt", "r")
print(file.read())
file.close()
```

Modules: A module is a Python file containing reusable code.

```
import math
print(math.sqrt(25))
```

Python also provides packages and external libraries that can be installed and imported when required.